Media Insight Engine

Architecture Walkthrough

Three subsystems, explained line by line, with the outcome of each

1. Certificate-based database authentication - how the backend logs in to Postgres with a client certificate instead of a password (mutual TLS via Vault).

2. The MCP server - how the engine exposes its tools and agents over the Model Context Protocol, mounted inside the API process.

3. GitHub Actions - how CI tests every change and publishes Docker images on main.

Every claim below points at the exact file and lines it came from, so you can open the source next to this page and follow along.

Part 1 - Certificate-based DB authentication

In one sentence

Instead of logging in with a password, the backend presents a client certificate over an encrypted TLS connection. Postgres checks that the certificate was signed by a trusted CA and that its name (CN=mie) matches the database role - and that IS the login. The certificate is handed to the backend at startup by HashiCorp Vault and lives only in RAM.

The four moving parts

- **cert-init (one-shot)** generates the CA, the server cert and the client cert; seeds Vault.
- **Vault** the safe that stores the client cert + private key.
- **Postgres (db)** runs with TLS on and checks client certs instead of passwords.
- **Backend (server)** fetches its cert from Vault at startup and connects with it.

1. **cert-init mints a CA, a server cert and a client cert (CN=mie)**
   A one-shot container runs openssl. The client certificate's Common Name is 'mie' - this is the pivot of the whole scheme, because Postgres will map that CN straight to the 'mie' database role. -nodes leaves the key unencrypted so libpq never blocks on a passphrase prompt.

scripts/cert-init.sh:18-32 (generate, idempotent)

```
if [ ! -f "$SRC/ca.crt" ]; then
  openssl req -x509 -nodes -newkey rsa:4096 -days 3650 \
    -keyout "$SRC/ca.key" -out "$SRC/ca.crt" -subj "/CN=mie-dev-ca"
  # server cert: SAN=db so verify-full matches the "db" hostname
  openssl req -new -nodes -newkey rsa:2048 -keyout "$SRC/server.key" ...
  # client cert: CN must equal the Postgres role; cert auth maps CN -> user
  openssl req -new -nodes -newkey rsa:2048 \
    -keyout "$SRC/client.key" -out /tmp/client.csr -subj "/CN=mie"
fi
```

2. **cert-init places the server certs and seeds Vault with the client bundle**
   The server cert/key/CA are copied onto a shared volume for Postgres (the key is chowned to uid 999 and locked to 0600, or Postgres refuses to start). The client bundle is written into Vault's KV store, and a least-privilege token - read-only on exactly that one secret - is minted for the app.

scripts/cert-init.sh:44-56 (seed Vault + scoped token)

```
vault kv put secret/mie/db-client \
  ca.crt=@"$SRC/ca.crt" \
  client.crt=@"$SRC/client.crt" \
  client.key=@"$SRC/client.key"
vault policy write mie-db - <<EOF
path "secret/data/mie/db-client" { capabilities = ["read"] }
EOF
vault token create -policy=mie-db -ttl=168h -field=token \
  > "$SHARED/vault-token"
```

3. **Postgres comes up in TLS mode and uses a custom auth rulebook**
   docker-compose starts Postgres with ssl=on plus a server cert, key and CA, and points it at a custom pg_hba.conf. All DB connections can now be encrypted, and the server can request a client cert.

docker-compose.yml:37-48 (db command)

```
command:
  - postgres
  - -c
  - ssl=on
  - -c
  - ssl_cert_file=/etc/postgres/certs/server.crt
  - -c
  - ssl_key_file=/etc/postgres/certs/server.key
  - -c
  - ssl_ca_file=/etc/postgres/certs/ca.crt
  - -c
  - hba_file=/etc/postgres/pg_hba.conf
```

4. **The auth rule says 'use the certificate, not a password'**
   pg_hba.conf is Postgres's rulebook, read top to bottom. The hostssl line matches TLS connections to the mie database and authenticates them by client certificate (method 'cert'). clientcert=verify-full means the cert's CN must equal the role name. The plain host line below it is the non-TLS password fallback used only by local, non-Docker development.

postgres/pg_hba.conf:6-9

```
# TYPE     DATABASE  USER  ADDRESS  METHOD
local      all       all            trust
hostssl    mie       mie   all      cert clientcert=verify-full
host       mie       mie   all      scram-sha-256   # non-TLS fallback
```

5. **The backend is told to use cert auth, with no password in the URL**
   The server container runs with DB_AUTH=cert and a DATABASE_URL that has a username but NO password - the certificate is the credential. /run/mie-certs is a tmpfs mount, so the private key the app fetches lives in RAM and never touches disk.

docker-compose.yml:64-75 (server environment + tmpfs)

```
environment:
  DB_AUTH: cert                      # mutual TLS
  DATABASE_URL: postgresql://mie@db:5432/mie   # no password
  VAULT_ADDR: http://vault:8200
  VAULT_TOKEN_FILE: /certs-shared/vault-token
  VAULT_DB_CERT_PATH: mie/db-client
  DB_CERT_DIR: /run/mie-certs
tmpfs:
  - /run/mie-certs                   # key/cert live in RAM only
```

6. **At startup the backend decides cert-vs-password and makes the connection**
   init_db() sees db_auth == 'cert' and calls secrets.ensure_db_certs(), which returns the libpq SSL parameters. Those are spread into psycopg.connect for the schema bootstrap, and into the connection pool so every pooled connection is a mutual-TLS connection.

server/app/db.py:17-28 (the decision + connect + pool)

```
ssl = secrets.ensure_db_certs() if settings.db_auth == "cert" else {}
with psycopg.connect(settings.database_url,
                     autocommit=True, **ssl) as conn:
    conn.execute(schema)
_pool = ConnectionPool(settings.database_url, min_size=1, max_size=10,
    open=True,
    kwargs={"autocommit": True, "row_factory": dict_row, **ssl},
    configure=register_vector)
```

7. **ensure_db_certs() reads the bundle from Vault and writes it 0600 to tmpfs**
   It reads the KV v2 secret over HTTP (the read path has a 'data/' infix and values live under .data.data), writes ca.crt / client.crt / client.key into the tmpfs dir at 0600 (libpq rejects a world-readable key), and maps each filename to its libpq parameter with sslmode=verify-full.

server/app/secrets.py:16-17, 48-60

```
_CERT_FILES = {"ca.crt": "sslrootcert",
               "client.crt": "sslcert",
               "client.key": "sslkey"}
def ensure_db_certs() -> dict[str, str]:
    data = _read_secret()            # HTTP GET /v1/secret/data/<path>
    ssl = {"sslmode": "verify-full"}
    for filename, param in _CERT_FILES.items():
        path = cert_dir / filename
        path.write_text(data[filename])
        path.chmod(0o600)            # libpq rejects world-readable key
        ssl[param] = str(path)
    return ssl
```

8. **The TLS handshake authenticates both sides - no password crosses the wire**
   sslmode=verify-full means the backend ALSO verifies the server's identity (correct CA + hostname 'db', which is why the server cert carried SAN=db). Postgres validates the client cert against the CA, reads CN=mie, maps it to the mie role via the hostssl rule, and authorizes the connection.

Outcome

In the containerized path the backend authenticates to Postgres with a short-lived client certificate delivered by Vault and held only in RAM - there is no DB password anywhere in the repo, the compose file or the connection string. The channel is encrypted both ways, and both ends prove their identity. A non-TLS password rule is kept purely so the local non-Docker workflow still works (it must connect with sslmode=disable to match the 'host' rule).

Part 2 - The MCP server

In one sentence

The engine exposes its tools and agents over the Model Context Protocol so an MCP client (Claude Desktop, Claude Code) can operate it. The server is built with the official 'mcp' SDK and mounted INSIDE the FastAPI process, served over streamable-HTTP at /mcp - so it shares the API's database pool, validation and error handling instead of re-implementing them.

1. **Build the server and give it operating instructions**
   An MCPServer is created once at import. The INSTRUCTIONS string is handed to the client as the server's system-level guidance (discover collections, which tool answers questions, and that long-running work is start-then-poll).

server/app/mcp_server.py:54

```
mcp = MCPServer("media-insight-engine", instructions=INSTRUCTIONS)
```

2. **Two helpers make every tool behave like the HTTP API**
   _guard calls a FastAPI router function and translates its HTTPException into an MCP ToolError, so HTTP status codes surface cleanly to the client. _dump coerces the router's return through the same Pydantic response model the HTTP API uses, so output is JSON-safe (dates become ISO strings).

server/app/mcp_server.py:57-69

```
def _guard(fn, *args, **kwargs):
    try:
        return fn(*args, **kwargs)
    except HTTPException as e:
        raise ToolError(f"{e.status_code}: {e.detail}") from e

def _dump(model, value):
    if isinstance(value, list):
        return [model.model_validate(v).model_dump(mode="json") ...]
    return model.model_validate(value).model_dump(mode="json")
```

3. **Each tool is a decorated function that calls the matching router**
   @mcp.tool registers the function as an MCP tool; its description is what the client reads to decide when to call it. The body builds the request model, calls the router through _guard, and dumps the result. Because it calls the ROUTER (not the raw service), it inherits persistence, validation and the insights 'stale' flag for free. Handlers are plain sync 'def' - mcp 2.x runs them on a worker thread, so blocking DB/Claude calls never stall the event loop.

server/app/mcp_server.py:74-81 (example tool)

```
@mcp.tool(description="Answer a question using ONLY a collection's
          sources, with citations ...")
def ask_collection(collection_id, question, history=None,
                   filters=None) -> dict:
    body = AskRequest(question=question, history=history or [],
                      filters=filters or AskFilters())
    return _dump(AskResponse,
        _guard(ask_router.ask, body=body, collection_id=collection_id))
```

The full tool set, grouped: Agents - ask_collection, get_collection_insights, refresh_collection_insights, run_question_bank, get_bank_run. Browse - list_collections, list_sources, get_source, get_sync_log. Manage - sync_collection, create/list/delete question_bank, get_bank_report, list/create/delete note.

4. **The server is exposed as an ASGI app over streamable-HTTP**
   streamable_http_app() turns the MCP server into an ASGI app. Passing streamable_http_path='/' makes the mount point itself the endpoint (rather than a nested sub-path), which is what lets it mount cleanly at /mcp.

server/app/mcp_server.py:193

```
mcp_app = mcp.streamable_http_app(streamable_http_path="/")
```

5. **FastAPI mounts it and runs its session manager in the app lifespan**
   main.py mounts the ASGI app at /mcp. A subtlety: FastAPI does NOT run a mounted app's own lifespan, so the streamable-HTTP session manager is entered explicitly inside the main app's lifespan context - otherwise the MCP transport would never start.

server/app/main.py:23-24, 43

```
# inside lifespan():
async with mcp.session_manager.run():
    yield
...
# MCP server over streamable-HTTP at /mcp
app.mount("/mcp", mcp_app)
```

6. **Connecting a client**
   The endpoint comes up with the API (docker compose up). Point any MCP client at http://localhost:8003/mcp.

server/docs/mcp.md (Claude Code CLI)

```
claude mcp add --transport http \
  media-insight-engine http://localhost:8003/mcp
```

Two gotchas documented in mcp.md: (1) the endpoint canonicalizes to /mcp/ with a trailing slash - a request to /mcp returns a 307 redirect that clients follow automatically; (2) requests must carry a Host of localhost:<port> or 127.0.0.1:<port> (built-in DNS-rebinding protection) or the server returns 421.

Outcome

A single process serves both the REST API and a fully-featured MCP server at /mcp. An MCP client gets ~18 tools - the engine's agents (ask, insights, question banks), browse tools and write/management tools - each one backed by the exact same router code, Pydantic models and error handling as the HTTP API, with zero duplicated business logic. Auth is none by design: localhost-only, matching the API's posture.

Part 3 - GitHub Actions CI/CD

In one sentence

One workflow file runs the server and client test suites on every pull request and every push to main; and on main only, after both test jobs pass, it builds and publishes the server and client Docker images to the GitHub Container Registry (GHCR).

1. **Triggers - run only when this project changes**
   The workflow fires on pull_request and on push to main, but both are path-filtered to media-insight-engine/** (plus the workflow file itself). Changes elsewhere in the monorepo don't trigger it.

.github/workflows/media-insight-engine.yml:4-9

```
on:
  pull_request:
    paths: [media-insight-engine/**, .github/workflows/...yml]
  push:
    branches: [main]
    paths: [media-insight-engine/**, .github/workflows/...yml]
```

2. **Concurrency and least-privilege permissions**
   concurrency cancels an in-progress run when a newer commit lands on the same ref, so stale runs don't pile up. The top-level permissions grant the token read-only access by default; the publish job widens this locally where it needs to.

.github/workflows/media-insight-engine.yml:11-16

```
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
permissions:
  contents: read
```

3. **Job 'server' - spin up a real Postgres, then run pytest**
   The job runs a pgvector service container with a health check, and points DATABASE_URL at it. Note DB_AUTH stays password here: no Vault or client certs in CI, so it uses the plain password path from Part 1. Env vars point at the repo's config/prompts/golden dirs and pin the embedding model to the one the Dockerfile bakes in.

.github/workflows/media-insight-engine.yml:24-43 (services + env)

```
services:
  db:
    image: pgvector/pgvector:pg17
    env: { POSTGRES_DB: mie, POSTGRES_USER: mie,
           POSTGRES_PASSWORD: ci-only }
    options: --health-cmd "pg_isready -U mie -d mie" ...
env:
  DATABASE_URL: postgresql://mie:ci-only@localhost:5432/mie
  EMBED_MODEL: BAAI/bge-small-en-v1.5
  FASTEMBED_CACHE_PATH: ${{ github.workspace }}/.cache/fastembed
```

4. **Job 'server' steps - install, cache, test**
   Checkout, set up Python 3.12 with pip caching, install requirements, install Chromium (one test renders PDFs with Playwright), cache the embedding model between runs, then run pytest. The golden set is deliberately NOT run here - it needs ANTHROPIC_API_KEY and makes real model calls.

.github/workflows/media-insight-engine.yml:45-59 (steps)

```
- uses: actions/checkout@v5
- uses: actions/setup-python@v6  # python 3.12, cache: pip
- run: pip install -r requirements.txt
- run: python -m playwright install --with-deps chromium
- uses: actions/cache@v4         # key: fastembed-<model>
- run: pytest -q
```

5. **Job 'client' - install, test, build**
   A parallel job for the React client: checkout, set up Node 22 with npm caching, npm ci, run the test suite, and build (which runs tsc --noEmit then vite build, so a type error fails CI).

.github/workflows/media-insight-engine.yml:66-75 (steps)

```
- uses: actions/checkout@v5
- uses: actions/setup-node@v5    # node 22, cache: npm
- run: npm ci
- run: npm test
- run: npm run build             # tsc --noEmit + vite build
```

6. **Job 'publish' - gated on main, after both test jobs pass**
   needs: [server, client] makes it wait for both and run only if they succeed. The if-condition restricts it to pushes on refs/heads/main, so pull requests never publish. It locally grants packages: write (needed to push to GHCR) on top of the read-only default.

.github/workflows/media-insight-engine.yml:77-85

```
publish:
  needs: [server, client]
  if: github.event_name == 'push'
      && github.ref == 'refs/heads/main'
  permissions:
    contents: read
    packages: write
```

7. **Job 'publish' steps - log in to GHCR and build/push both images**
   Log in to ghcr.io with the built-in GITHUB_TOKEN, set up Buildx, then build and push the server and client images. Each image is tagged twice - with the commit SHA (immutable) and latest - and uses the GitHub Actions cache (type=gha) to speed up layer reuse. Images are only built on main because the server image bakes in Chromium and the 1.6 GB Whisper model.

.github/workflows/media-insight-engine.yml:89-104 (server image)

```
- uses: docker/login-action@v3   # registry ghcr.io
    password: ${{ secrets.GITHUB_TOKEN }}
- uses: docker/setup-buildx-action@v3
- uses: docker/build-push-action@v6
    context: media-insight-engine/server
    push: true
    tags: |
      ${{ REGISTRY }}/mie-server:${{ github.sha }}
      ${{ REGISTRY }}/mie-server:latest
    cache-from/to: type=gha,scope=mie-server
```

Outcome

Every PR and every push to main gets the server's pytest suite (against a real pgvector database) and the client's tests + type-checked build run automatically, with path filters, caching and auto-cancellation keeping runs fast and cheap. When a change lands on main and both suites are green, two freshly-built Docker images (mie-server and mie-client) are published to GHCR, each tagged with the commit SHA and latest - ready to pull and deploy.

Where to look - file map

DB certificate auth

- **scripts/cert-init.sh** the openssl bootstrap that mints the CA, server and client certs + seeds Vault.
- **docker-compose.yml** wires Vault, cert-init, Postgres (ssl=on) and the server (DB_AUTH=cert).
- **postgres/pg_hba.conf** the auth rule: hostssl ... cert clientcert=verify-full.
- **server/app/db.py** chooses cert vs password and makes the connection / pool.
- **server/app/secrets.py** fetches the cert bundle from Vault, returns the libpq SSL params.

MCP server

- **server/app/mcp_server.py** the MCPServer, the tool handlers and the ASGI app.
- **server/app/main.py** mounts the MCP app at /mcp and runs its session manager.
- **server/docs/mcp.md** endpoint, connect instructions and the trailing-slash / Host gotchas.

GitHub Actions

- **.github/workflows/media-insight-engine.yml** the whole CI/CD pipeline: server + client tests, publish to GHCR.